

Branch-and-Cut Redistricting with Auditable Solver Reports

Gio Della-Libera

Draft — May 11, 2026

Abstract

This draft describes the BISECT branch-and-cut implementation contract for redistricting. The paper is an exact-optimization implementation paper: it separates formulation, separation metadata, solver status, fallback behavior, and RPLAN/RCTX audit lineage so exact claims require proof or bound metadata. It treats fallback and timeout reports as first-class outputs rather than failures to be hidden behind a valid final plan.

1 Introduction

U.16 documents the ‘bisect-ilp’ and ‘bisect-cli’ branch-and-cut surface: ‘–structure ilp –ilp-method branch-and-cut’. Its claim is not that every redistricting instance is solved exactly at scale, but that exact-optimization runs should expose formulation, separation, proof status, and audit lineage in a reproducible form.

2 Implementation Contract

The implementation contract distinguishes formulation-only paths, active branch-and-cut paths, solver fallback paths, and completed proof/bound paths. Each run summary should name the model, objective, constraint profile, separation routine status, solve status, incumbent plan source, and any proof or bound metadata.

The report is the boundary between implementation success and exactness. A run can be successful as software because it produced a valid audited plan, while still failing to support an exact optimization claim because the solver timed out, fell back, or never produced a bound. U.16 therefore requires every output to carry a status label before any exactness language is used.

3 Audit Artifacts

The final-plan handoff is the RPLAN/RCTX fixed point. A branch-and-cut run can emit a valid RPLAN plus solver report, manifest, and audit certificate. The certificate verifies

Field	Purpose	Failure mode avoided
Formulation id	Names variables and constraints	Ambiguous model claim
Cut summary	Records separation activity	Hidden non-branch-and-cut path
Solver status	Distinguishes solved, timed out, fallback	False exactness
Proof/bound	Carries exact evidence when available	Valid plan mistaken for proof

Table 1: Minimum branch-and-cut report fields.

Status	Meaning	Allowed claim
Formulation-only	Model built, no active separation proof	reproducible model contract
Cut-active	Separation ran with cut metadata	branch-and-cut behavior observed
Bounded	Solver reports incumbent and bound	scoped bound under model
Proven	Solver reports proof under assumptions	scoped exact optimum/infeasibility
Fallback	Heuristic or prior plan returned	valid audited fallback only

Table 2: Status vocabulary for branch-and-cut outputs.

declared profile and lineage; it does not prove optimality unless the solver report also carries the relevant proof or bound.

The audit package should retain both the plan artifact and the exact-method claim artifact. RPLAN/RCTX records the final assignment and context, while the solver report records whether separation, bounds, proof, timeout, or fallback occurred. Dropping either side creates ambiguity: a valid plan without a solver report is not an exact result, and a solver report without an auditable final plan is not a reusable BISECT output.

4 Evaluation Plan

The current evidence ladder is fixture and staged exact-contract evidence. Required L0/L1 checks include formulation construction, cut-report emission, fallback status clarity, and RPLAN sidecar verification. Real-data exact performance and broad runtime claims remain future benchmark evidence.

Claim	Current evidence	Publication gap
Formulation reports are reproducible	L0 formulation fixtures	command transcript table
Fallback status is explicit	L0/L1 status tests	failure-mode examples
RPLAN sidecars verify lineage	L1 audit package checks	public example package
Exact performance scales	future benchmark placeholder	real-data benchmark protocol

Table 3: Evidence ladder for U.16 claims.

5 Limitations

Exact optimization claims are scoped to the model, data, objective, constraints, and solver status. A valid final plan is not an optimality proof. Large instances may time out or fall back, and those outcomes must be reported as such rather than normalized away.

The current draft should be read as an implementation-contract paper. It does not yet provide a public real-data solve table, broad runtime evidence, or a claim that branch-and-cut dominates heuristic construction. Those claims require benchmark commands, solver versions, instance metadata, and archived outputs.

6 Conclusion

U.16 turns branch-and-cut from a black-box exact option into an auditable implementation contract. The paper’s next readiness step is a simulated panel review focused on proof semantics, fallback reporting, and solver-report reproducibility.

Once those semantics are stable, branch-and-cut can sit beside heuristic and construction methods without category confusion: exact claims live in the solver report, validity and

lineage claims live in the audit package, and future performance claims live in benchmark evidence.

References

BISECT project. U.16 branch-and-cut redistricting paper plan, 2026. Local planning document and implementation roadmap.